

Práctica: Creación de un Chat Multi-Modelo con Interfaz Gráfica

Objetivo: Modificar una aplicación existente en Python con Gradio para integrar múltiples Grandes Modelos de Lenguaje (LLMs) y permitir al usuario elegir con cuál interactuar mediante un selector.

1. Descripción del Escenario

Actualmente disponemos de scripts que conectan con una sola IA. El objetivo de esta práctica es unificar esas conexiones en una única interfaz "Universal". Los alumnos deberán implementar la lógica de control (backend) y los componentes visuales (frontend) necesarios para alternar entre proveedores como OpenAI, Google y Anthropic dinámicamente.

2. Requisitos Previos

- Tener instalado Python y VS Code (o editor similar).
- Haber instalado las librerías necesarias:

```
pip install gradio openai google-generativeai anthropic python-dotenv
```

- Disponer de un archivo .env con las claves de API válidas para al menos tres proveedores.

3. Instrucciones de la Práctica

Paso 1: Configuración de Seguridad

Crea un archivo llamado multichat.py. Lo primero que debes implementar es la carga segura de credenciales.

- **Tarea:** Importa dotenv y os. Configura el script para que lea las claves API (OPENAI_API_KEY, GOOGLE_API_KEY, etc.) desde vuestro archivo .env local.
- **Restricción:** Está terminantemente prohibido escribir las claves directamente en el código ("hardcoded").

Paso 2: Modularización de Funciones

Debes crear tres funciones independientes. Cada una debe recibir un texto (prompt) y devolver una cadena de texto (string) con la respuesta de la IA.

1. `consultar_gpt(prompt)`: Debe conectar con un modelo de la familia GPT (ej. gpt-3.5-turbo o gpt-4).
2. `consultar_gemini(prompt)`: Debe conectar con Google Gemini. **Nota:** Asegúrate de usar un modelo vigente (ej. gemini-2.0-flash o gemini-1.5-flash).
3. `consultar_claude(prompt)`: Debe conectar con la API de Anthropic (ej. claude-3-opus o sonnet).

Paso 3: Lógica del Selector ("El Router")

Implementa una función controladora llamada `generar_respuesta(prompt, modelo_seleccionado)`.

- Esta función debe recibir la elección del usuario desde la interfaz gráfica.
- Debe usar una estructura condicional (`if/elif/else`) para llamar a la función específica según el modelo elegido.
- **Manejo de errores:** Si el usuario no escribe nada o selecciona un modelo no válido, la función debe devolver un mensaje de error amigable.

Paso 4: Diseño de la Interfaz (GUI)

Utilizando la librería **Gradio**, construye una interfaz que cumpla con los siguientes requisitos visuales:

1. **Entrada:** Un cuadro de texto (Textbox) de varias líneas para la pregunta.
2. **Selector:** Un menú desplegable (Dropdown) que liste claramente las opciones: "ChatGPT", "Gemini" y "Claude". Por defecto debe haber uno seleccionado.
3. **Salida:** Un área de respuesta que interprete formato **Markdown**. Esto es vital para que si la IA genera código o tablas, se vean correctamente (negritas, bloques de código, etc.).
4. **Botones:** Un botón de "Enviar" y otro de "Limpiar".

Paso 5: Ejecución y Prueba

Lanza la aplicación. Debes demostrar que puedes hacer la misma pregunta (por ejemplo: *"¿Cómo hago un bucle for en Python?"*) a los tres modelos simplemente cambiando el valor del selector, sin reiniciar el programa.

4. Criterios de Evaluación (Rúbrica)

Criterio	Excelente (10)	Aceptable (5-8)	Insuficiente (<5)
Funcionalidad	Los 3 modelos responden correctamente y se puede cambiar entre ellos fluidamente.	Uno de los modelos falla o el selector no actualiza la lógica.	El código no ejecuta o solo funciona un modelo fijo.
Interfaz (UI)	Usa Markdown correctamente, tiene etiquetas claras y el Dropdown funciona.	La salida es texto plano (sin formato) o la interfaz es confusa.	No hay selector o faltan componentes básicos.
Código Limpio	Uso correcto de funciones separadas y gestión de claves con <code>.env</code> .	Código espagueti (todo junto) o claves escritas en el código.	Código desordenado que impide su lectura.